

C2-G Alex 和 Marvolo 不会出题了- 3

C2-G Alex 和 Marvolo 不会出题了-3

题面

G Alex 和 Marvolo 不会出题了-3

题目描述

输入格式

输出格式

样例输入

样例输出

题解

思路详解

时间复杂度

参考代码 (C)

题面

G Alex 和 Marvolo 不会出题了-3

时间限制：1000ms 内存限制：65536kb

题目描述

Alex 和 Marvolo 不会出题了。于是他们开始闲聊：

Alex: 感觉他们现在的警觉性很高。

Marvolo: 同感，上一道题的 hint 已经成为表情包了。

Alex: 我决定金盆洗手，改邪归正。出一点有难度有挑战性的算法题

Marvolo: 哦吼？

Alex: 你平时看电视嘛？

Marvolo: 不怎么看，我都玩手机的。

Alex: 对，电视上面的节目都是限时的，你过了这个点就看不到了。

Marvolo: 手机上看不是就行了。

Alex: no no no，一般来说电视上是最早播出来的，电视放完了手机上才更新。

Marvolo: 所以呢？

Alex:（翻了一会儿书包，掏出一张密密麻麻的纸）这里有所有电视台的节目单，包括每个节目的起始时间和结束时间。你说如果我时间充裕，我一天能看完多少节目？

Marvolo: 唔，晚上 23:00 我追的番才更新，所以我一天只看一个。

简而言之，一共有 $n(1 \leq n \leq 106)$ 个节目，分别在不同的电台上播放。给定每个节目的起始时间和结束时间 $s_i, e_i, (1 \leq s_i, e_i \leq 105)$ ，假设时间无限，最多可以看完多少个不同的节目呢？看完一个节目是指，从这个节目开始的时间，到它结束的时间，都在观看这个节目。

输入格式

第一行一个正整数 $n(1 \leq n \leq 106)$

接下来的 n 行，每行 2 个正整数 $s_i, e_i(1 \leq s_i < e_i \leq 105)$ ，表示一个节目的起止时间。

输出格式

输出一行一个正整数，表示最多可以看完的节目数量

样例输入

```
3
1 3
3 4
1 4
```

样例输出

```
2
```

题解

思路详解

对于这道题来说，很明显我们要使用贪心算法来进行解答，贪心算法是指在对问题求解是，总是做出当前看来是最好的选择。贪心算法不是对所有问题都能得到整体最优，关键是贪心策略的选择，选择贪心的策略必须具备无后效性，即某个状态以前的过程不会影响以后的状态，只与当前状态有关。

那么这道题我们应该如何选择看电视的策略呢？那就是先看最先能看完的，既保证了最短时间内能看完一个节目，又能让下次看节目的时间最早。所以说呢，我们需要先将所有的电视节目按照结束时间进行排序。这里建议大家使用快排或者是归并排序，毕竟数据量比较大，我们的时间有限。

```
void merge(struct map x[ ],struct map tmp[ ],int left,int leftend,int rightend)
{
    int i=left, j=leftend+1, q=left;
    while(i<=leftend && j<=rightend)
    {
        if(x[i].over<x[j].over)

            tmp[q++]=x[i++];

        else if(x[i].over==x[j].over)
        {
            if(x[i].start<=x[j].start)
                tmp[q++]=x[i++];
            else
                tmp[q++]=x[j++];
        }
        else
        {
            tmp[q++]=x[j++];
        }
    }
    while(i<=leftend)
        tmp[q++]=x[i++];
    while(j<=rightend)
        tmp[q++]=x[j++];
    for(i=left; i<=rightend; i++)
        x[i]=tmp[i];
}
```

然后就是使当前时间为 0，从排序好的第一个节目开始寻找，如果当前节目能够完整观看，就使时间跳跃到当前节目的结束时间，能看的节目数+1，判断下一个节目能否观看；如果不能看，直接判断下一个节目能否完整观看。只到判断完最后一个节目。

```
for(i=0,time=0;i<n;i++){
    if(time<=map[i].start){
        time=map[i].over;
        count++;
    }
}
```

时间复杂度

在这道题里，我们用到了归并排序和从 1 到 n 的遍历，因此时间复杂度为 $O(n\log n)$

参考代码 (C)

```
#include <stdio.h>
struct map
{
    int start;
    int over;
}map[1000006];
void merge(struct map x[ ],struct map tmp[ ],int left,int leftend,int rightend)
{
    int i=left, j=leftend+1, q=left;
    while(i<=leftend && j<=rightend)
    {
        if(x[i].over<x[j].over)

            tmp[q++]=x[i++];

        else if(x[i].over==x[j].over)
        {
            if(x[i].start<=x[j].start)
                tmp[q++]=x[i++];
            else
                tmp[q++]=x[j++];
        }
        else
        {
            tmp[q++]=x[j++];
        }
    }
    while(i<=leftend)
        tmp[q++]=x[i++];
    while(j<=rightend)
        tmp[q++]=x[j++];
    for(i=left; i<=rightend; i++)
        x[i]=tmp[i];
}
void mergeSort(struct map k[ ],int n)
{
    int *tmp;
    tmp = (struct map *)malloc(sizeof(struct map) * n);
    if(tmp != NULL) {
```

```

        mSort(k, tmp, 0, n-1);
        free(tmp);
    }
}

void mSort(struct map k[], struct map tmp[], int left, int right)
{
    int center;
    if(left < right){
        center = (left+right)/2;
        mSort(k, tmp, left, center);
        mSort(k, tmp, center+1, right);
        merge(k, tmp, left, center, right);
    }
}

int main()
{
    int n,i,count=0,time=0;
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d%d",&map[i].start,&map[i].over);
    mergeSort(map,n);
    for(i=0,time=map[0].start;i<n;i++)
    {
        if(time<=map[i].start)
        {
            time=map[i].over;
            count++;
        }
    }
    printf("%d",count);
}

```